

# 编译原理-语法分析 05

## 自底向上语法分析

授课教师：柯嵩宇

福州大学计算机与大数据学院

2025 年 4 月 3 日



福州大学  
FUZHOU UNIVERSITY 1958



## ① 课前复习

## ② 自底向上的语法分析



## ① 课前复习

## ② 自底向上的语法分析

## 编译过程的主要步骤：

- ① 词法分析
  - 输入：字符流；
  - 输出：词元流、符号表；
- ② 语法分析
  - 输入：词元流、符号表；
  - 输出：语法树、符号表；
- ③ 语义分析
  - 输入：语法树、符号表；
  - 输出：修饰后的语法树；
- ④ 中间代码生成
  - 输入：修饰后的语法树；
  - 输出：中间表示 IR；
- ⑤ 机器无关优化
  - 输入：IR；
  - 输出：IR；
- ⑥ 机器相关优化
  - 输入：IR；
  - 输出：IR；
- ⑦ 代码生成
  - 输入：IR；
  - 输出：汇编、字节码；

## ① 词法分析

- 输入：字符流；
- 输出：词元流、符号表；

## ② 语法分析

- 输入：词元流、符号表；
- 输出：语法树、符号表；

### ③ 语义分析

- 输入：语法树、符号表；
- 输出：修饰后的语法树；

#### ④ 中间代码生成

- 输入: 修饰后的语法树;
- 输出: 中间表示 IR;

## ⑤ 机器无关优化

- 输入: IR;
- 输出: IR;

## ⑥ 机器相关优化

- 输入: IR;
- 输出: IR;

## ⑦ 代码生成

- 输入：IR；
- 输出：汇编、字节码；

# 词法分析器

## 正则文法

正则文法  $\mathcal{G} = (\mathbb{N}, \Sigma, \mathbb{P}, S)$ ，其中文法

- 产生式规则集  $\mathbb{P}$ ，每个规则都为如下形式

$$\mathbb{N} \rightarrow \Sigma^* \mathbb{N},$$

即正则文法的产生式左侧仅允许存在一个非终结符，右侧终结符必须在非终结符前面。

- 不是一般性地，我们可以要求只有一个非终结符在右侧的产生式中。

# 上下文无关文法

上下文无关文法  $\mathcal{G} = (\mathbb{N}, \Sigma, \mathbb{P}, S)$ ，其中文法

- 产生式规则集  $\mathbb{P}$ ，每个规则都为如下形式

$$\mathbb{N} \rightarrow (\Sigma \cup \mathbb{N})^*,$$

即上下文无关文法的产生式左侧仅允许存在一个非终结符。

# 扩展知识之泵引理

在可计算性理论中的形式语言理论中，泵引理（**Pumping lemma**）声称给定类的任何语言可以被“抽吸”并仍属于这个类。<sup>1</sup>

其核心意思是对于超过某个特定长度的字符串，可以通过重复其中的某些片段来构造同属于该语言的更长的字符串。

用途：泵引理用于证明某些语言不是正则语言/上下文无关语言。

<sup>1</sup><https://zh.wikipedia.org/wiki/泵引理>

# 扩展知识之泵引理

## 引理 (正则语言的泵引理)

$L$  是正则语言, 则存在整数  $n \geq 1$  使得任意长度超过  $n$  的字符串  $w$  都可以被改写成  $xyz$  的形式并满足<sup>2</sup>:

- ①  $|xy| \leq n$ ;
- ②  $|y| \geq 1$ ;
- ③  $\forall k \geq 0 : xy^kz \in L$ .

<sup>2</sup> $n$  为泵长度, 也叫做泵常数, 可理解为正则语言等效的极小化 DFA 的状态个数

# 扩展知识之泵引理

## 引理 (正则语言的泵引理)

$L$  是正则语言, 则存在整数  $n \geq 1$  使得任意长度超过  $n$  的字符串  $w$  都可以被改写成  $xyz$  的形式并满足<sup>2</sup>:

- ①  $|xy| \leq n$ ;
- ②  $|y| \geq 1$ ;
- ③  $\forall k \geq 0 : xy^kz \in L$ .

如何证明?

<sup>2</sup> $n$  为泵长度, 也叫做泵常数, 可理解为正则语言等效的极小化 DFA 的状态个数

# 扩展知识之泵引理

## 正则语言的泵引理

$\{0^n 1^n | n \geq 1\}$  不是正则语言.

假设  $\{0^n 1^n | n \geq 1\}$  是正则，则存在  $p$  (泵引理常数) 使得泵引理成立。

取字符串  $w = 0^p 1^p$ ，尝试将其划分为三个不重叠的子串  $x, y, z$ ，使其满足泵引理的条件： $|xy| \leq p \wedge |y| \geq 1 \wedge \forall k \geq 0 : xy^k z \in L$ 。

会发现无论如何划分， $xy^0 z = xz$  都会应为 0 的数量比 1 少而不属于  $L$ 。

这与泵引理冲突，因此  $\{0^n 1^n | n \geq 1\}$  不是正则语言。



# 扩展知识之泵引理

## 引理 (上下文无关语言的泵引理)

$L$  是下文无关语言, 则存在整数  $n \geq 1$  使得任意长度超过  $n$  的字符串  $w$  都可以被改写成  $uvxyz$  的形式并满足:

- ①  $|vxy| \leq n$ ;
- ②  $|vy| \geq 1$ ;
- ③  $\forall k \geq 0 : uv^kxy^kz \in L$ .

# 自顶向下 (Top-Down) 语法分析

## 特点:

- 从**开始符号**出发, 寻找“匹配”的**推导**。
- 推导: 根据文法的产生式规则, 把串中出现的产生式的左部符号替换成右部。
- 从**根**开始, 构造语法树。
- 具体实现: 递归下降分析法、预测分析法。

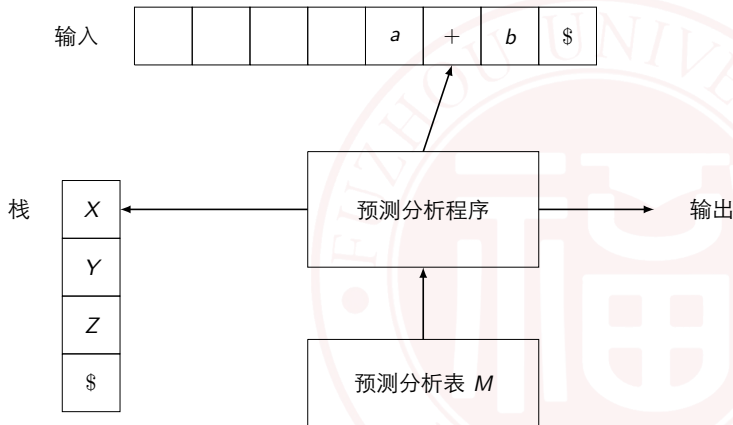
## 表驱动的预测分析法步骤:

- ① 处理文法: 左递归、回溯、二义性;
- ② 计算 FIRST 和 FOLLOW;
- ③ 计算预测分析表  $M$ ;
- ④ 计算同步集合用于错误处理等;
- ⑤ 处理输入。

# 表驱动的预测分析法

## 定义

- **预测分析法**是一种表驱动的方法，它由下推栈，预测分析表和控制程序组成，是一种确定性的下推自动机。
- 特征：不需要为每个非终结符编写递归下降过程，而是根据预测分析表构造一个不需要回溯的非递归下降语法分析器。
- 要求：文法是 LL(1) 的。只有对于 LL(1) 文法，可以对输入串进行有效的无回溯的自上而下分析。



一个分析表驱动的分析器模型

# 预测分析表

## 定义

- **预测分析表**是用于自顶向下语法分析（如 LL(1) 分析器）的核心工具，它通过非终结符和输入符号的匹配，指导分析器选择正确的产生式，避免回溯。
- 预测分析表是一个二维表格（矩阵）：
  - 行代表每一个非终结符；

# 预测分析表

## 定义

- **预测分析表**是用于自顶向下语法分析（如 LL(1) 分析器）的核心工具，它通过非终结符和输入符号的匹配，指导分析器选择正确的产生式，避免回溯。
- 预测分析表是一个二维表格（矩阵）：
  - 行代表每一个非终结符；
  - 列代表每一个终结符（包括结束符号 \$）；

# 预测分析表

## 定义

- **预测分析表**是用于自顶向下语法分析（如 LL(1) 分析器）的核心工具，它通过非终结符和输入符号的匹配，指导分析器选择正确的产生式，避免回溯。
- 预测分析表是一个二维表格（矩阵）：
  - 行代表每一个非终结符；
  - 列代表每一个终结符（包括结束符号 \$）；
  - **每个单元格里填写对应的产生式，或者错误标记；**

# 预测分析表

## 定义

- **预测分析表**是用于自顶向下语法分析（如 LL(1) 分析器）的核心工具，它通过非终结符和输入符号的匹配，指导分析器选择正确的产生式，避免回溯。
- 预测分析表是一个二维表格（矩阵）：
  - 行代表每一个非终结符；
  - 列代表每一个终结符（包括结束符号 \$）；
  - 每个单元格里填写对应的产生式，或者错误标记；
  - 单元格  $(i, j)$  的内容表示若在匹配第  $i$  行的非终结符时遇到第  $j$  列的终结符时，该确定性地选用的生成式。

# 预测分析表

## Algorithm 1: 预测分析表构造算法

**Input:**  $\mathbb{P}$ ,  $\text{FIRST}(\cdot)$ ,  $\text{FOLLOW}(\cdot)$

```
1 for  $A \rightarrow \alpha \in \mathbb{P}$  do
2   if  $a \in \text{FIRST}(\alpha)$  then
3      $M[A, a] \leftarrow (A \rightarrow \alpha)$ ;
4   end
5   if  $\varepsilon \in \text{FIRST}(\alpha)$  then
6     for  $b \in \text{FOLLOW}(A)$  do
7        $M[A, b] \leftarrow (A \rightarrow \alpha)$ ;
8     end
9   end
10 end
11 剩余空格打上错误标记;
12 如果一个格子里面有多于 1 条规则, 不是 LL(1) 文法, 返回错误;
```

# 表驱动的预测分析法

## Algorithm 2: 表驱动的预测分析法

**Input:** 输入串  $w$ , 文法  $\mathcal{G}$  的预测分析表  $M$

**Output:** 如果  $w \in \mathcal{L}(\mathcal{G})$ , 输出  $w$  的一个最左推导, 否则输出错误提示

```

1  $X \Leftarrow$  栈顶符号;  $ip$  指向当前处理的输入符号;
2 while  $X \neq \$$  /* 栈非空 */ do
3   if  $X = *ip$  then
4      $\text{pop}(X); ip++;$ 
5   else if  $X \in \mathbb{V}_T$  or  $M[X, *ip]$  是一条报错条目 then
6     错误处理;
7   else if  $M[X, *ip] = X \rightarrow Y_1, Y_2, \dots, Y_k$  then
8     输出  $X \rightarrow Y_1, Y_2, \dots, Y_k;$ 
9      $\text{pop}(X);$ 
10     $\text{push}(Y_k); \text{push}(Y_{k-1}); \dots; \text{push}(Y_1);$ 
11  end
12 end
```

## 练习

对下列文法构建预测分析表并分析  $(a, (a, a))$ :

$$S \rightarrow (L) \mid a$$

$$L \rightarrow L, S \mid S$$

## 练习

对下列文法构建预测分析表并分析  $(a, (a, a))$ :

$$S \rightarrow (L) \mid a$$

$$L \rightarrow L, S \mid S$$

### ① 消除左递归:

设定文法符号的顺序为  $S, L$ 。

$S$  无左递归, 无需处理。

将  $S$  代入  $L$  得:  $L \rightarrow L, S \mid (L) \mid a$  消除  $L$  的左递归:

$$L \rightarrow (L)L' \mid aL' \quad L' \rightarrow, SL' \mid \epsilon$$

得到最终文法

$$S \rightarrow (L) \mid a \quad L \rightarrow (L)L' \mid aL' \quad L' \rightarrow, SL' \mid \epsilon$$

# 练习

$$S \rightarrow (L)|a \quad L \rightarrow (L)L'|aL' \quad L' \rightarrow ,SL'|\epsilon$$

## ③ 计算 FIRST:

- $\text{FIRST}(S) = \{a, ( \}$
- $\text{FIRST}(L) = \{a, ( \}$
- $\text{FIRST}(L') = \{ , , \epsilon \}$

## ④ 计算 FOLLOW:

如果存在一个产生式  $A \rightarrow \alpha B \beta$ , 则  $\text{FIRST}(\beta)$  中除了  $\epsilon$  以外的符号都在  $\text{FOLLOW}(B)$  中。  
 如果存在一个产生式  $A \rightarrow \alpha B$ , 或存在  $A \rightarrow \alpha B \beta \wedge \epsilon \in \text{FIRST}(\beta)$ , 则  $\text{FOLLOW}(A)$  所有符号都在  $\text{FOLLOW}(B)$  中。

# 练习

$$S \rightarrow (L)|a \quad L \rightarrow (L)L'|aL' \quad L' \rightarrow ,SL'|\epsilon$$

## ③ 计算 FIRST:

- $\text{FIRST}(S) = \{a, ( \}$
- $\text{FIRST}(L) = \{a, ( \}$
- $\text{FIRST}(L') = \{ , , \epsilon \}$

## ④ 计算 FOLLOW:

- $\text{FOLLOW}(S) = \{ \$, ) \}$
- $\text{FOLLOW}(L) = \{ \}$
- $\text{FOLLOW}(L') = \{ \}$

如果存在一个产生式  $A \rightarrow \alpha B \beta$ , 则  $\text{FIRST}(\beta)$  中除了  $\epsilon$  以外的符号都在  $\text{FOLLOW}(B)$  中。  
如果存在一个产生式  $A \rightarrow \alpha B$ , 或存在  $A \rightarrow \alpha B \beta \wedge \epsilon \in \text{FIRST}(\beta)$ , 则  $\text{FOLLOW}(A)$  所有符号都在  $\text{FOLLOW}(B)$  中。

# 练习

$$S \rightarrow (L)|a \quad L \rightarrow (L)L'|aL' \quad L' \rightarrow ,SL'|\epsilon$$

- $\text{FIRST}(S) = \{a, (\}$
- $\text{FIRST}(L) = \{a, (\}$
- $\text{FIRST}(L') = \{, , \epsilon\}$
- $\text{FOLLOW}(S) = \{\$, )\}$
- $\text{FOLLOW}(L) = \{)\}$
- $\text{FOLLOW}(L') = \{)\}$

- ① 若终结符  $a \in \text{FIRST}(\alpha)$ , 将  $A \rightarrow \alpha$  加入  $M[A, a]$
- ② 若  $\epsilon \in \text{FIRST}(\alpha)$ , 则  $\forall b \in \text{FOLLOW}(A)$ , 将  $A \rightarrow \alpha$  加入  $M[A, b]$
- ⑤ 构建预测分析表:

|    | a                   | ,                     | (                     | )                         | \$ |
|----|---------------------|-----------------------|-----------------------|---------------------------|----|
| S  | $S \rightarrow a$   |                       | $S \rightarrow (L)$   |                           |    |
| L  | $L \rightarrow aL'$ |                       | $L \rightarrow (L)L'$ |                           |    |
| L' |                     | $L' \rightarrow ,SL'$ |                       | $L' \rightarrow \epsilon$ |    |

分析  $(a, (a, a))$

# 预测分析法

## 错误恢复

### 如何检测错误？

- ① 栈顶的终结符号与输入不匹配；

### 如何处理错误？

- ① 恐慌模式 (Panic mode)：忽略输入流中的一系列符号，直到出现符合设计者预期的词元；
- ② 短语恢复：改变、插入或删除输入中的符号，并给出适当的错误信息。

# 预测分析法

## 错误恢复

### 如何检测错误？

- ① 栈顶的终结符号与输入不匹配；
- ② 栈顶的非终结符号  $A$ ，但  $M[A, a] = error$ 。

### 如何处理错误？

- ① 恐慌模式 (Panic mode)：忽略输入流中的一系列符号，直到出现符合设计者预期的词元；
- ② 短语恢复：改变、插入或删除输入中的符号，并给出适当的错误信息。

# 恐慌模式的错误处理

## 定义

**同步集合 (synchronization set)**: 错误处理函数/过程会不断丢弃输入的符号直到遇到一个同步集合内的符号。

如何构造同步集合?

- ① 将 FOLLOW( $A$ ) 里的元素都加入同步集合;

# 恐慌模式的错误处理

## 例

针对文法：

`assign_stmt := expression ;`

输入：`a+b+`;

因为  $;\in \text{FOLLOW}(\text{expression})$ ，在其同步集合中，故尝试丢弃 `+` 后继续分析。

够了吗？

## 例 (反例)

`a = 100 if (b != a) ...`

# 恐慌模式的错误处理

## 定义

**同步集合 (synchronization set)**: 错误处理函数/过程会不断丢弃输入的符号直到遇到一个同步集合内的符号。

如何构造同步集合?

- ① 将 FOLLOW( $A$ ) 里的元素都加入同步集合;
- ② 将更高层次的非终结符号的 FOLLOW 集合也加入同步集合中;
- ③ 将 FIRST( $A$ ) 的符号也加入同步集合;
- ④ 如果非终结符可以生成  $\epsilon$ , 则使用它作为默认值;
- ⑤ 直接把不匹配的终结符号弹出;

# 语法分析器的类型

## 自顶向下 (Top-Down)

- 从开始符号出发，寻找“匹配”的推导。
- 推导：根据文法的产生式规则，把串中出现的产生式的左部符号替换成右部。
- 从根开始，构造语法树。
- 具体实现：递归下降分析法、预测分析法。

## 自底向上 (Bottom-Up)

- 从输入串开始，逐步进行归约，直到文法的开始符号。
- 归约：根据文法的产生式规则，把串中出现的产生式的右部替换成左部符号。
- 从叶节点开始，构造语法树。
- 具体实现：算符优先分析法、LR 分析法。

## 短语、句柄、前缀

# 自底向上 (Bottom-Up) 语法分析

- 从分析树的底部(叶节点) 向顶部(根节点) 方向构造分析树，可以看成是将输入串 $w$  **归约**为文法开始符号 $S$ 的过程。
- 自顶向下的语法分析采用**最左推导**方式。
- 自底向上的语法分析采用**最左归约**方式（反向构造**最右推导**）。

## 定义 (归约, Reduce)

**归约**：先将一个子串与某规则的右部匹配，然后将该子串替换为该规则的左部。

# 自底向上 (Bottom-Up) 语法分析

## 核心问题

① 什么时候进行归约？归约到什么程度？

例

a

# 自底向上 (Bottom-Up) 语法分析

## 核心问题

- ① 什么时候进行归约？归约到什么程度？

例

$a+b$

# 自底向上 (Bottom-Up) 语法分析

## 核心问题

- ① 什么时候进行归约？归约到什么程度？

例

$a+b*c$

# 自底向上 (Bottom-Up) 语法分析

## 核心问题

- ① 什么时候进行归约？归约到什么程度？

例

`a+b*c`

- ② 选择哪个产生式进行归约？

例

`int x=5;`



## ① 课前复习

## ② 自底向上的语法分析

移入-规约框架

短语、句柄、前缀

# 自底向上 (Bottom-Up) 语法分析

## 通用框架

### 定义 (移入-归约分析 (Shift-Reduce Parsing))

- 在对输入串的一次从左到右扫描过程中, 语法分析器将零个或多个输入符号移入到栈的顶端, 直到它可以对栈顶的一个文法符号串  $\beta$  进行归约为止;
- 然后, 它将  $\beta$  归约为某个产生式的左部;
- 语法分析器不断地重复这个循环, 直到它检测到一个语法错误, 或者栈中包含了开始符号且输入缓冲区为空<sup>3</sup>为止。

<sup>3</sup>当进入这样的格局时, 语法分析器停止运行, 并宣称成功完成了语法分析。

# 移入-归约分析

例

给定文法：

$$S \rightarrow aAcBe$$
$$A \rightarrow b$$
$$A \rightarrow Ab$$
$$B \rightarrow d$$

试分析 *abbcde*

# 移入-归约分析

例

给定文法：

$$S \rightarrow aAcBe$$

$$A \rightarrow b$$

$$A \rightarrow Ab$$

$$B \rightarrow d$$

试分析 *abbcde*

最左规约——最右推导的逆过程：

$$S \Rightarrow aAcBe \Rightarrow aAcde \Rightarrow aAbcde \Rightarrow abbcde$$

# 移入-归约分析

移入-归约分析器可采取的 4 种动作：

- ① 移入：将下一个输入符号移到栈的顶端。
- ② 归约：被归约的符号串的**右端**必然处于栈顶。语法分析器在栈中确定这个串的**左端**，并决定用哪个非终结符来替换这个串。
- ③ 接受：宣布语法分析过程成功完成。
- ④ 报错：发现一个语法错误，并调用错误恢复子例程。

# 移入-归约分析

## 挑战

### 例

给定文法：

$$S \rightarrow aAcBe$$

$$A \rightarrow b$$

$$A \rightarrow Ab$$

$$B \rightarrow d$$

当栈内元素为  $aAb$  时调用哪一条规则进行规约？

## 短语、句柄、前缀

# 基本概念

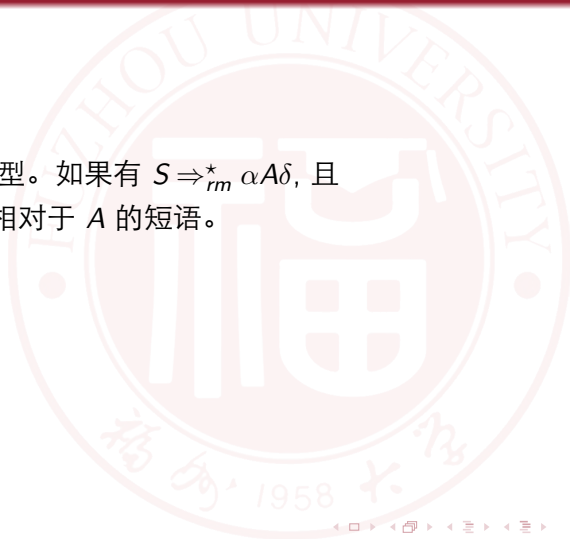
对于文法  $G(S)$ ,  $\alpha\beta\delta$  是  $G$  的一个句型。



# 基本概念

对于文法  $G(S)$ ,  $\alpha\beta\delta$  是  $G$  的一个句型。如果有  $S \Rightarrow_{rm}^* \alpha A \delta$ , 且

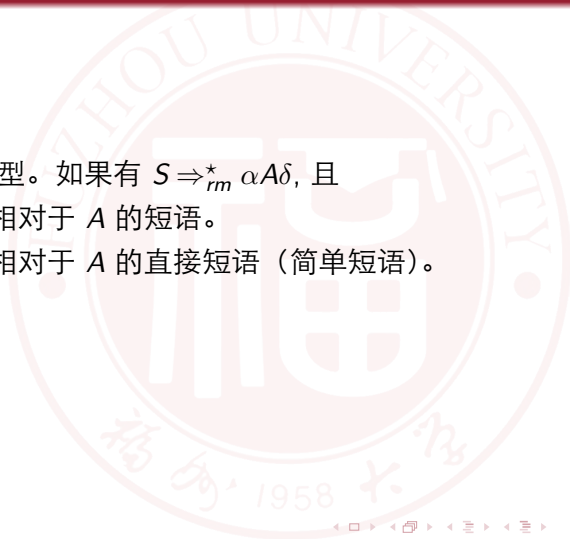
- $A \Rightarrow_{rm}^+ \beta$ , 则称  $\beta$  是句型  $\alpha\beta\delta$  相对于  $A$  的短语。



# 基本概念

对于文法  $G(S)$ ,  $\alpha\beta\delta$  是  $G$  的一个句型。如果有  $S \Rightarrow_{rm}^* \alpha A \delta$ , 且

- $A \Rightarrow_{rm}^+ \beta$ , 则称  $\beta$  是句型  $\alpha\beta\delta$  相对于  $A$  的短语。
- $A \Rightarrow_{rm} \beta$ , 则称  $\beta$  是句型  $\alpha\beta\delta$  相对于  $A$  的直接短语（简单短语）。



# 基本概念

对于文法  $G(S)$ ,  $\alpha\beta\delta$  是  $G$  的一个句型。如果有  $S \Rightarrow_{rm}^* \alpha A \delta$ , 且

- $A \Rightarrow_{rm}^+ \beta$ , 则称  $\beta$  是句型  $\alpha\beta\delta$  相对于  $A$  的短语。
- $A \Rightarrow_{rm} \beta$ , 则称  $\beta$  是句型  $\alpha\beta\delta$  相对于  $A$  的直接短语（简单短语）。
- 一个句型的最左直接短语称为该句型的句柄（Handle）。

# 经典例子

## 例

用下列文法分析  $i_1 + i_2 * i_3$ :

$$E \rightarrow T \mid E + T$$

$$T \rightarrow F \mid T * F$$

$$F \rightarrow (E) \mid i$$

$$\begin{aligned} E &\Rightarrow E + T \Rightarrow E + T * F \Rightarrow E + T * i_3 \\ &\Rightarrow E + F * i_3 \Rightarrow E + i_2 * i_3 \Rightarrow T + i_2 * i_3 \\ &\Rightarrow F + i_2 * i_3 \Rightarrow i_1 + i_2 * i_3 \end{aligned}$$

对于句型  $i_1 + i_2 * i_3$  而言,

- 短语有  $i_1, i_2, i_3, i_2 * i_3, i_1 + i_2 * i_3$ ;
- 直接短语有  $i_1, i_2, i_3$ ;
- 句柄是  $i_1$ 。

# 经典例子

## 例

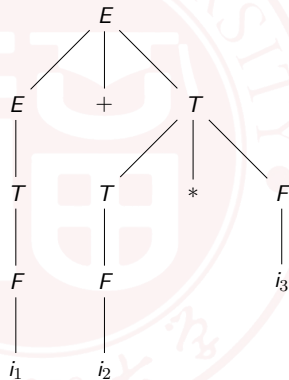
用下列文法分析  $i_1 + i_2 * i_3$ :

$$E \rightarrow T \mid E + T$$

$$T \rightarrow F \mid T * F$$

$$F \rightarrow (E) \mid i$$

$$\begin{aligned} E &\Rightarrow E + T \Rightarrow E + T * F \Rightarrow E + T * i_3 \\ &\Rightarrow E + F * i_3 \Rightarrow E + i_2 * i_3 \Rightarrow T + i_2 * i_3 \\ &\Rightarrow F + i_2 * i_3 \Rightarrow i_1 + i_2 * i_3 \end{aligned}$$

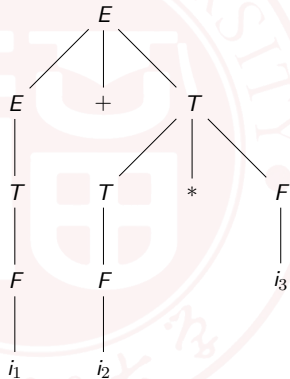


# 经典例子

## 例

在一个句型的语法树中

- 任一子树的叶节点从左到右排列所组成的符号串都是该句型的短语。

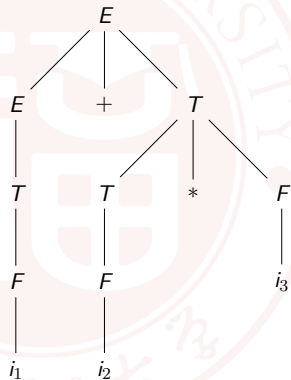


# 经典例子

## 例

在一个句型的语法树中

- 任一子树的叶节点从左到右排列所组成的符号串都是该句型的短语。
- 任一最小子树（树高?为 1）的叶节点所组成的符号串都是该句型的直接短语。

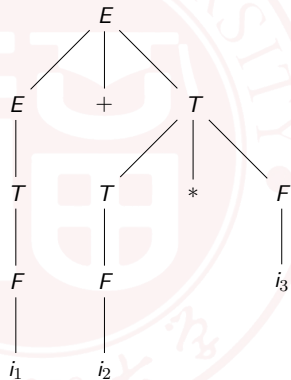


# 经典例子

## 例

在一个句型的语法树中

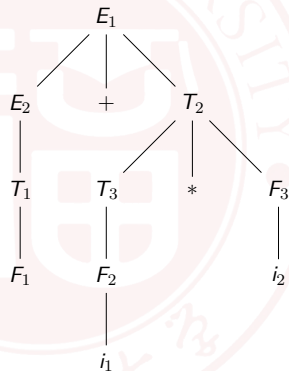
- 任一子树的叶节点从左到右排列所组成的符号串都是该句型的短语。
- 任一最小子树（树高?为 1）的叶节点所组成的符号串都是该句型的直接短语。
- 句柄是最左边的直接短语。



# 经典例子

## 例

在句型  $F_1 + i_1 * i_2$  中

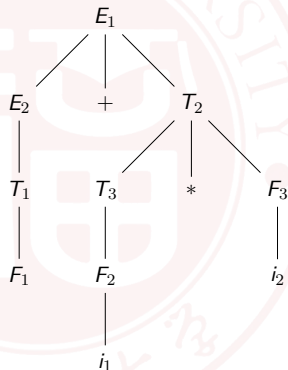


# 经典例子

## 例

在句型  $F_1 + i_1 * i_2$  中

- $F_1$  是相对  $T_1$  和  $E_2$  的短语，是相对  $T_1$  的直接短语。

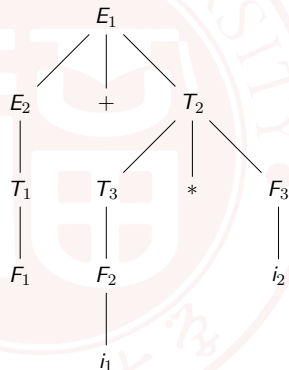


# 经典例子

## 例

在句型  $F_1 + i_1 * i_2$  中

- $F_1$  是相对  $T_1$  和  $E_2$  的短语，是相对  $T_1$  的直接短语。
- $i_1$  是相对  $F_2$  的直接短语，是相对  $F_2$  和  $T_3$  的短语。

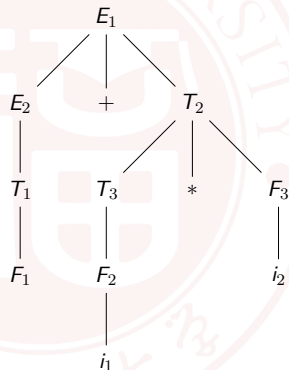


# 经典例子

## 例

在句型  $F_1 + i_1 * i_2$  中

- $F_1$  是相对  $T_1$  和  $E_2$  的短语，是相对  $T_1$  的直接短语。
- $i_1$  是相对  $F_2$  的直接短语，是相对  $F_2$  和  $T_3$  的短语。
- $i_2$  是相对  $F_3$  的直接短语。

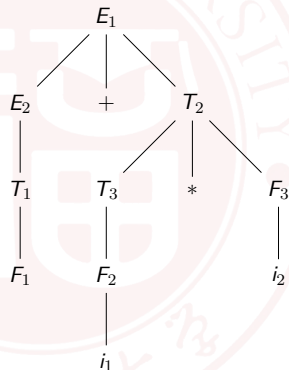


# 经典例子

## 例

在句型  $F_1 + i_1 * i_2$  中

- $F_1$  是相对  $T_1$  和  $E_2$  的短语，是相对  $T_1$  的直接短语。
- $i_1$  是相对  $F_2$  的直接短语，是相对  $F_2$  和  $T_3$  的短语。
- $i_2$  是相对  $F_3$  的直接短语。
- 句柄是

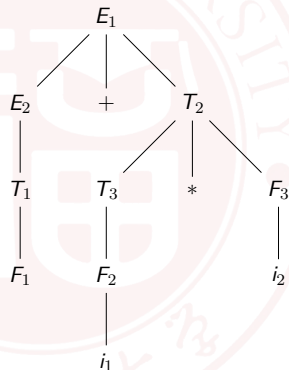


# 经典例子

## 例

在句型  $F_1 + i_1 * i_2$  中

- $F_1$  是相对  $T_1$  和  $E_2$  的短语，是相对  $T_1$  的直接短语。
- $i_1$  是相对  $F_2$  的直接短语，是相对  $F_2$  和  $T_3$  的短语。
- $i_2$  是相对  $F_3$  的直接短语。
- 句柄是  $F_1$ 。



# 练习题

## 练习

对于文法  $G(S) : S \rightarrow SS + \mid SS * \mid a$ , 求句型  $SSS + a * +$  的句柄。

# 练习题

## 练习

对于文法  $G(S) : S \rightarrow SS+ \mid SS* \mid a$ , 求句型  $SSS+a*+$  的句柄。

## 解

方法一：写出最右推导

$$S \Rightarrow SS+ \Rightarrow SSS*+ \Rightarrow S\textcolor{red}{S}a*+ \Rightarrow S\textcolor{red}{SS}+a*+$$

故  $SS+$  为该句型的句柄。

# 练习题

## 练习

对于文法  $G(S) : S \rightarrow SS+ \mid SS* \mid a$ , 求句型  $SSS+a*+$  的句柄。

## 解

方法一：写出最右推导

$$S \Rightarrow SS+ \Rightarrow SSS*+ \Rightarrow SSa*+ \Rightarrow SSS+a*+$$

故  $SS+$  为该句型的句柄。方法二：画出语法树（看黑板）

# 练习题

## 练习

对于文法  $G(S) : S \rightarrow SS+ \mid SS* \mid a$ , 求句型  $SSS+a*+$  的句柄。

## 解

方法一：写出最右推导

$$S \Rightarrow SS+ \Rightarrow SSS*+ \Rightarrow SSa*+ \Rightarrow SSS+a*+$$

故  $SS+$  为该句型的句柄。方法二：画出语法树（看黑板）

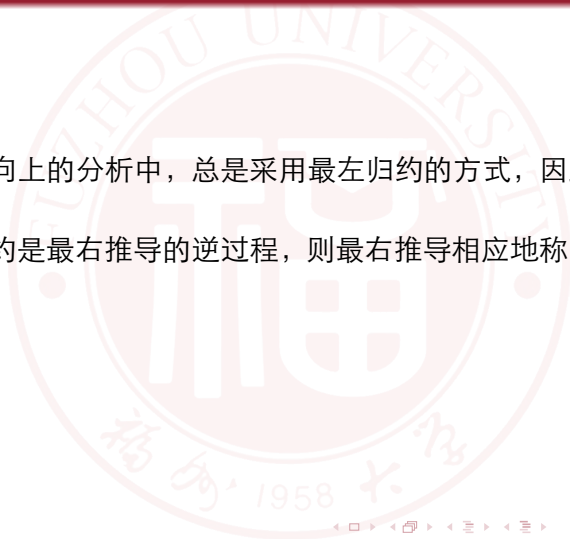
本句型还有哪些短语、直接短语？

## 规范句型

- **规范归约**  $\equiv$  **最左归约**：在自底向上的分析中，总是采用最左归约的方式，因此把最左归约称为规范归约。

# 规范句型

- **规范归约**  $\equiv$  **最左规约**：在自底向上的分析中，总是采用最左归约的方式，因此把最左归约称为规范归约。
- **规范推导**  $\equiv$  **规范推导**：规范归约是最右推导的逆过程，则最右推导相应地称为规范推导。



# 规范句型

- **规范归约**  $\equiv$  **最左规约**：在自底向上的分析中，总是采用最左归约的方式，因此把最左归约称为规范归约。
- **规范推导**  $\equiv$  **规范推导**：规范归约是最右推导的逆过程，则最右推导相应地称为规范推导。
- **规范句型**：由规范推导推出的句型称为规范句型。

# 前缀

- ① 前缀：符号串的任意首部。



# 前缀

- ① 前缀：符号串的任意首部。

例

$abc$  的前缀有  $\varepsilon, a, ab, abc$

# 前缀

- ① 前缀：符号串的任意首部。

## 例

$abc$  的前缀有  $\epsilon, a, ab, abc$

- ② 后缀？

# 前缀

- ① 前缀：符号串的任意首部。

## 例

$abc$  的前缀有  $\varepsilon, a, ab, abc$

- ② 后缀？
- ③ 活前缀：文法  $G(S)$  的一个规范句型  $\alpha\beta$ ，其中  $\beta$  为终结符号串。若  $\alpha$  不含句柄之后的任何符号，则称  $\alpha$  为**活前缀（可行前缀）**。

# 前缀

- ① 前缀：符号串的任意首部。

## 例

$abc$  的前缀有  $\varepsilon, a, ab, abc$

- ② 后缀？
- ③ 活前缀：文法  $G(S)$  的一个规范句型  $\alpha\beta$ ，其中  $\beta$  为终结符号串。若  $\alpha$  不含句柄之后的任何符号，则称  $\alpha$  为**活前缀（可行前缀）**。
- ④ 如果  $\alpha$  是含有句柄的活前缀，则称  $\alpha$  为**可归前缀**。

# 前缀

- ① 前缀：符号串的任意首部。

## 例

$abc$  的前缀有  $\varepsilon, a, ab, abc$

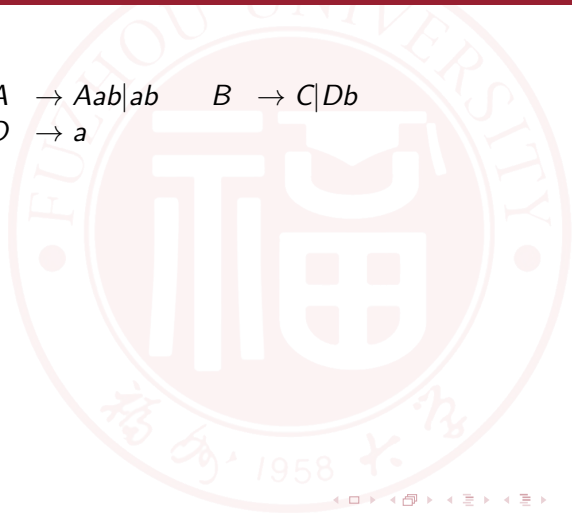
- ② 后缀？
- ③ 活前缀：文法  $G(S)$  的一个规范句型  $\alpha\beta$ ，其中  $\beta$  为终结符号串。若  $\alpha$  不含句柄之后的任何符号，则称  $\alpha$  为**活前缀（可行前缀）**。
- ④ 如果  $\alpha$  是含有句柄的活前缀，则称  $\alpha$  为**可归前缀**。
- ⑤ **可归前缀即最长的活前缀。**

# 例

已知文法  $G(S)$ :

$$\begin{array}{lll} S & \rightarrow & CbBA \\ C & \rightarrow & a \\ A & \rightarrow & Aab|ab \\ D & \rightarrow & a \\ B & \rightarrow & C|Db \end{array}$$

和句子  $ababab$



# 例

已知文法  $G(S)$ :

$$\begin{array}{lll} S \rightarrow CbBA & A \rightarrow Aab|ab & B \rightarrow C|Db \\ C \rightarrow a & D \rightarrow a & \end{array}$$

句子  $ababab$  的规范推导过程为:

$$S \Rightarrow CbBA \Rightarrow CbBab \Rightarrow CbDbab \Rightarrow Cbabab \Rightarrow ababab$$

- ① 句型  $ababab$  的活前缀是 , 可归前缀是 ;

# 例

已知文法  $G(S)$ :

$$\begin{array}{lll} S \rightarrow CbBA & A \rightarrow Aab|ab & B \rightarrow C|Db \\ C \rightarrow a & D \rightarrow a & \end{array}$$

句子  $ababab$  的规范推导过程为:

$$S \Rightarrow CbBA \Rightarrow CbBab \Rightarrow CbDbab \Rightarrow Cbabab \Rightarrow ababab$$

- ① 句型  $ababab$  的活前缀是 , 可归前缀是 ;

# 例

已知文法  $G(S)$ :

$$\begin{array}{lll} S \rightarrow CbBA & A \rightarrow Aab|ab & B \rightarrow C|Db \\ C \rightarrow a & D \rightarrow a & \end{array}$$

句子  $ababab$  的规范推导过程为:

$$S \Rightarrow CbBA \Rightarrow CbBab \Rightarrow CbDbab \Rightarrow Cbabab \Rightarrow ababab$$

- ① 句型  $ababab$  的活前缀是  $a$ , 可归前缀是  $a$ ;

# 例

已知文法  $G(S)$ :

$$\begin{array}{lll} S \rightarrow CbBA & A \rightarrow Aab|ab & B \rightarrow C|Db \\ C \rightarrow a & D \rightarrow a & \end{array}$$

句子  $ababab$  的规范推导过程为:

$$S \Rightarrow CbBA \Rightarrow CbBab \Rightarrow CbDbab \Rightarrow Cbabab \Rightarrow ababab$$

- ① 句型  $ababab$  的活前缀是  $a$ ，可归前缀是  $a$ ；
- ② 句型  $Cbabab$  的活前缀是 \_\_\_\_\_，可归前缀是 \_\_\_\_\_；

# 例

已知文法  $G(S)$ :

$$\begin{array}{lll} S \rightarrow CbBA & A \rightarrow Aab|ab & B \rightarrow C|Db \\ C \rightarrow a & D \rightarrow a & \end{array}$$

句子  $ababab$  的规范推导过程为:

$$S \Rightarrow CbBA \Rightarrow CbBab \Rightarrow CbDbab \Rightarrow Cbabab \Rightarrow ababab$$

- ① 句型  $ababab$  的活前缀是  $a$ ，可归前缀是  $a$ ;
- ② 句型  $Cbabab$  的活前缀是 \_\_\_\_\_，可归前缀是 \_\_\_\_\_;

# 例

已知文法  $G(S)$ :

$$\begin{array}{lll} S \rightarrow CbBA & A \rightarrow Aab|ab & B \rightarrow C|Db \\ C \rightarrow a & D \rightarrow a & \end{array}$$

句子  $ababab$  的规范推导过程为:

$$S \Rightarrow CbBA \Rightarrow CbBab \Rightarrow CbDbab \Rightarrow Cbabab \Rightarrow ababab$$

- ① 句型  $ababab$  的活前缀是  $a$ , 可归前缀是  $a$ ;
- ② 句型  $Cbabab$  的活前缀是  $C, Cb, Cba$ , 可归前缀是  $Cba$ ;

# 例

已知文法  $G(S)$ :

$$\begin{array}{lll} S \rightarrow CbBA & A \rightarrow Aab|ab & B \rightarrow C|Db \\ C \rightarrow a & D \rightarrow a & \end{array}$$

句子  $ababab$  的规范推导过程为:

$$S \Rightarrow CbBA \Rightarrow CbBab \Rightarrow CbDbab \Rightarrow Cbabab \Rightarrow ababab$$

- ① 句型  $ababab$  的活前缀是  $a$ , 可归前缀是  $a$ ;
- ② 句型  $Cbabab$  的活前缀是  $C, Cb, Cba$ , 可归前缀是  $Cba$ ;
- ③ 句型  $CbDbab$  的活前缀是 \_\_\_\_\_, 可归前缀是 \_\_\_\_\_;

# 例

已知文法  $G(S)$ :

$$\begin{array}{lll} S \rightarrow CbBA & A \rightarrow Aab|ab & B \rightarrow C|Db \\ C \rightarrow a & D \rightarrow a & \end{array}$$

句子  $ababab$  的规范推导过程为:

$$S \Rightarrow CbBA \Rightarrow CbBab \Rightarrow CbDbab \Rightarrow Cbabab \Rightarrow ababab$$

- ① 句型  $a$ babab 的活前缀是  $a$ , 可归前缀是  $a$ ;
- ② 句型  $Cb$  $a$ bab 的活前缀是  $C, Cb, Cba$ , 可归前缀是  $Cba$ ;
- ③ 句型  $CbD$  $b$ ab 的活前缀是  $CbD$ , 可归前缀是  $CbD$ ;

# 例

已知文法  $G(S)$ :

$$\begin{array}{lll} S \rightarrow CbBA & A \rightarrow Aab|ab & B \rightarrow C|Db \\ C \rightarrow a & D \rightarrow a & \end{array}$$

句子  $ababab$  的规范推导过程为:

$$S \Rightarrow CbBA \Rightarrow CbBab \Rightarrow CbDbab \Rightarrow Cbabab \Rightarrow ababab$$

- ① 句型  $a$  $babab$  的活前缀是 $a$ , 可归前缀是 $a$ ;
- ② 句型  $Cb$  $a$  $bab$  的活前缀是 $C,Cb,Cba$ , 可归前缀是 $Cba$ ;
- ③ 句型  $CbD$  $bab$  的活前缀是 $C,Cb,CbD,CbDb$ , 可归前缀是 $CbDb$ ;